

Programação Funcional

Listas e Casamento de Padrões em Haskell

Vladimir Oliveira Di Iorio
DPI - UFV

Padrões em Haskell

- constantes inteiras, booleanas e caracteres;
- variáveis;
- tuplas de padrões; ex: $(p1, p2, \dots)$ onde $p1, p2, \dots$ também são padrões.
- listas de padrões; ex: $(p1:p2)$ onde $p1$ e $p2$ também são padrões.
- padrão `"_"`;
- construtores de padrões.

Padrões em Haskell

- Quando um argumento a casa com um padrão p ?
 - se p é uma constante e $p=a$;
 - se p é uma variável x , sendo que a será associado a x ;
 - se p é uma tupla $(p_1, p_2, \dots)$, $a=(a_1, a_2, \dots)$ e cada componente de a casa com o respectivo componente de p .
 - se p é uma lista $(p_1:p_2)$, a é uma lista não vazia e a cabeça de a casa com p_1 e sua cauda casa com p_2 .
 - se o padrão p for `"_"`.

Padrões em Haskell

- Restrição importante: não é permitido usar um mesmo nome mais de uma vez dentro do mesmo padrão!
- Exemplo:

```
1 func :: Int -> Int -> Bool
2 func x x = True
3 func x y = False
```

ERRO - variável repetida em padrão!!!

Construtor case

```
1 numToString :: Int -> String
2 numToString x =
3     case x of
4     0 -> "zero"
5     1 -> "um"
6     2 -> "dois"
7     3 -> "três"
8     4 -> "quatro"
9     5 -> "cinco"
10    6 -> "seis"
11    7 -> "sete"
12    8 -> "oito"
13    9 -> "nove"
14    otherwise -> ""
```

Diferentes Definições p/ funções

- Definir uma função

`sumPairs :: [(Int,Int)] -> Int`

que calcula a soma total dos pares de inteiros de uma lista.

```
Main> sumPairs [(2,3), (5,-1), (3,3)]  
15
```

Caso Base

- Função

`sumPairs :: [(Int,Int)] -> Int`

que calcula a soma total dos pares de inteiros de uma lista.

```
1 sumPairs :: [(Int,Int)] -> Int
2 sumPairs [] = 0
3 ... ???
```

Possíveis Soluções

```
1 sumPairs :: [(Int,Int)] -> Int
2 sumPairs [] = 0
3 sumPairs (a:x) = fst a + snd a + sumPairs x
```

```
1 sumPairs :: [(Int,Int)] -> Int
2 sumPairs [] = 0
3 sumPairs (a:x) = c + d + sumPairs x
4                 where
5                 c = fst a
6                 d = snd a
```

Possíveis Soluções

```
1 sumPairs :: [(Int,Int)] -> Int
2 sumPairs [] = 0
3 sumPairs (a:x) = c + d + sumPairs x
4                 where
5                 c = fst a
6                 d = snd a
```

```
1 sumPairs :: [(Int,Int)] -> Int
2 sumPairs [] = 0
3 sumPairs (a:x) = c + d + sumPairs x
4                 where
5                 (c,d) = a
```

Possíveis Soluções

```
1 sumPairs :: [(Int,Int)] -> Int
2 sumPairs [] = 0
3 sumPairs (a:x) = c + d + sumPairs x
4                 where
5                 (c,d) = a
```

```
1 sumPairs :: [(Int,Int)] -> Int
2 sumPairs [] = 0
3 sumPairs ((c,d):x) = c + d + sumPairs x
```

Possíveis Soluções

```
1 sumPairs :: [(Int,Int)] -> Int
2 sumPairs [] = 0
3 sumPairs ((c,d):x) = c + d + sumPairs x
```

```
1 sumPairs :: [(Int,Int)] -> Int
2 sumPairs [] = 0
3 sumPairs (a:x) = sumPair a + sumPairs x
4 sumPair (c,d) = c + d
```

Diferentes Definições p/ funções

- Definir uma função

`zip :: [Int] -> [Int] -> [(Int,Int)]`

que agrupa duas listas de inteiros em uma única lista de pares.

```
Main> zip [1,2,3] [4,6,1]
[(1,4), (2,6), (3,1)]
Main> zip [1,3] [1,5,8]
[(1,1), (3,5)]
Main> zip [1,3] [2]
[(1,2)]
```

Possíveis Soluções

```
1 zip :: [Int] -> [Int] -> [(Int,Int)]
2 zip (a:x) (b:y) = (a,b) : zip x y
3 zip x [] = []
4 zip [] y = []
```

```
1 zip :: [Int] -> [Int] -> [(Int,Int)]
2 zip (a:x) (b:y) = (a,b) : zip x y
3 zip _ _ = []
```

Aplicação: BD para Biblioteca

- Uma biblioteca usa um banco de dados para manter um registro dos livros emprestados aos clientes.
- *Pessoas e livros* serão representados por Strings.
- O BD será uma lista de pares (Pessoa, Livro).
- Se um par (p, b) estiver no BD, isso significa que o cliente p pegou um exemplar do livro b emprestado.

Tipos

```
1 type Pessoa = String
2 type Livro = String
3 type BancoDados = [ (Pessoa, Livro) ]
```

- Exemplo de BD:

```
exBD = [ ("Alice", "Senhor dos Anéis"),
         ("Ana", "Windows 3.1"),
         ("Alice", "Excel para Leigos"),
         ("Ricardo", "Senhor dos Anéis") ]
```

Funções de pesquisa

- As funções de pesquisa em BD recebem como entrada um BD, uma pessoa ou livro, e retornam o resultado da pesquisa.
- Exemplos:
 - Dada uma pessoa, encontrar os livros emprestados por ela.
 - Dado um livro (pode ter mais de uma cópia), listar as pessoas que estão com o mesmo emprestado.
 - Dado um livro, descobrir se está emprestado.
 - Dada uma pessoa, calcular o número de livros que ela tem emprestados no momento.

Funções de pesquisa

- As funções de pesquisa em BD recebem como entrada um BD, uma pessoa ou livro, e retornam o resultado da pesquisa.
- Exemplos:

```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 clientes :: BancoDados -> Livro -> [Pessoa]
3 emprestado :: BancoDados -> Livro -> Bool
4 numEmprest :: Bancodados -> Pessoa -> Int
```

Funções de atualização

- As funções de atualização recebem como entrada um BD e retornam um novo BD, resultado de uma operação.
- Exemplos:
 - Emprestar um livro a um cliente.
 - Devolver um livro emprestado.

```
1  fazEmprest :: BancoDados -> Pessoa -> Livro
2              -> BancoDados
3  retEmprest  :: BancoDados -> Pessoa -> Livro
4              -> BancoDados
```

Definindo as funções de pesquisa

- Definir a função

`livros :: BancoDados -> Pessoa -> [Livro]`

Exemplo:

```
exBD = [ ("Alice", "Senhor dos Anéis"),  
         ("Ana", "Windows 3.1"),  
         ("Alice", "Excel para Leigos"),  
         ("Ricardo", "Senhor dos Anéis") ]
```

```
Main> livros exBD "Alice"  
["Senhor dos Anéis", "Excel para Leigos"]  
Main> livros exBD "Ronaldo"  
[]
```

Definindo as funções de pesquisa

- Definir a função

`livros :: BancoDados -> Pessoa -> [Livro]`

```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 livros [] p = ???
3 livros ((n,b) : resto) p = ???
4
5
```

Definindo as funções de pesquisa

- Definir a função

`livros :: BancoDados -> Pessoa -> [Livro]`

```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 livros [] p = []
3 livros ((n,b) : resto) p = ???
4
5
```

Definindo as funções de pesquisa

- Definir a função

`livros :: BancoDados -> Pessoa -> [Livro]`

```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 livros [] p = []
3 livros ((n,b) : resto) p
4   | p == n      = ???
5   | otherwise   = ???
```

Definindo as funções de pesquisa

- Definir a função

`livros :: BancoDados -> Pessoa -> [Livro]`

```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 livros [] p = []
3 livros ((n,b) : resto) p
4   | p == n      = b : livros resto p
5   | otherwise   =      livros resto p
```

- As funções `clientes`, `emprestado`, `numEmprest` podem ser definidas de forma semelhante.

Definindo as funções de atualização

- Definir a função

```
fazEmprest :: BancoDados -> Pessoa -> Livro  
            -> BancoDados
```

- Basta adicionar um novo par ao banco de dados (pode haver repetições).

```
1 fazEmprest :: BancoDados -> Pessoa -> Livro  
2             -> BancoDados  
3 fazEmprest db p b = (p,b) : db
```

Definindo as funções de atualização

- Definir a função

```
retEmprest :: BancoDados -> Pessoa -> Livro  
            -> BancoDados
```

- Pesquisar o BD e remover a primeira ocorrência do par (pes, liv).
- Se não existir esse par no BD, é caracterizada uma situação de erro!

Definindo as funções de atualização

```
1  retEmprest :: BancoDados -> Pessoa -> Livro
2              -> BancoDados
3
4  retEmprest ((p,b) : resto) pes liv
5      | p == pes && b == liv  = ???
6      | otherwise           = ???
7
8  retEmprest [] pes liv
9      = error ("falha em retEmprest:"
10             ++ pes ++ " " ++ liv)
```

Definindo as funções de atualização

```
1  retEmprest :: BancoDados -> Pessoa -> Livro
2              -> BancoDados
3
4  retEmprest ((p,b) : resto) pes liv
5    | p == pes && b == liv = resto
6    | otherwise          = (p,b) : retEmprest resto pes liv
7
8  retEmprest [] pes liv
9    = error ("falha ao emprestar livro")
10
```

Cancela apenas a primeira
ocorrência do par.

Definindo as funções de atualização

- Definição alternativa, cancelando todas as ocorrências do par:

```
1  retEmprest :: BancoDados -> Pessoa -> Livro
2              -> BancoDados
3
4  retEmprest ((p,b) : resto) pes liv
5    | p==pes && b==liv  = retEmprest resto pes liv
6    | otherwise       = (p,b) : retEmprest resto pes liv
7
8  retEmprest [] pes liv = []
```

Testando

- Uso de "\$\$" :

```
Main> fazEmprest exBD "Alice" "Prog. Funcional"  
[ ("Alice", "Prog. Funcional"),  
  ("Alice", "Senhor dos Anéis"),  
  ("Ana", "Windows 3.1"),  
  ("Alice", "Excel para Leigos"),  
  ("Ricardo", "Senhor dos Anéis") ]  
Main> retEmprest $$ "Ana" "Windows 3.1"  
[ ("Alice", "Prog. Funcional"),  
  ("Alice", "Senhor dos Anéis"),  
  ("Alice", "Excel para Leigos"),  
  ("Ricardo", "Senhor dos Anéis") ]
```

List Comprehensions

- Maneira alternativa para construir e manipular listas.
- Útil para construir listas baseadas em outras listas.
- Exemplo: se a lista x é $[1, 3, 8]$ então

$[2*a \mid a \leftarrow x]$

será

$[2, 6, 16]$

- O exemplo acima pode ser lido como: construa uma lista onde cada elemento tem a forma $2*a$, sendo que a vem da lista x .

List Comprehensions

- Uma list comprehension `a <- x` é chamada **gerador**.
- Do lado esquerdo de "`<-`" sempre se coloca um padrão (ex:variável), e do lado direito, uma lista.

- Geradores podem ser combinados com testes:

`[ 2*a | a <- x, isEven a, a>3 ]`

se `x` é `[2,6,3,8]`, produz como resposta:

`[12,16]`

(supor `isEven` retorna `True` se argumento for par).

Exemplos

- Exemplo:

```
1 addPairs :: [(Int,Int)] -> [Int]
2 addPairs pL = [ a+b | (a,b) <- pL ]
```

```
Main>addPairs [(2,3), (4,3), (1,7)]
[5,7,8]
```

Exemplos

- Exemplo com teste:

```
1 newAddPairs :: [(Int,Int)] -> [Int]
2 newAddPairs pL = [ a+b | (a,b) <- pL, a<b ]
```

```
Main>newAddPairs [(2,3), (4,3), (1,7)]
[5,8]
```

Exemplos

- Redefinindo exemplos anteriores:

```
1 double :: [Int] -> [Int]
2 double [] = []
3 double (a:x) = (2*a) : double x
```

```
1 double :: [Int] -> [Int]
2 double l = [ 2*a | a <- l ]
```

Exemplos

- Redefinindo exemplos anteriores:

```
1 digits :: String -> String
2 digits [] = []
3 digits (a:x)
4   | isDigit a      = a : digits x
5   | otherwise      = digits x
```

```
1 digits :: String -> String
2 digits st = [ ch | ch <- st, isDigit ch ]
```

Exemplo usando BD

```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 livros [] p = []
3 livros ((n,b) : resto) p
4   | p == n      = b : livros resto p
5   | otherwise   =      livros resto p
```

```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 livros db p =
3   [ liv | (pes,liv) <- db, pes==p ]
```

Erro de lógica comum

```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 livros db p =
3   [ liv | (pes,liv) <- db, pes==p ]
```

argumento p

- versão errada:

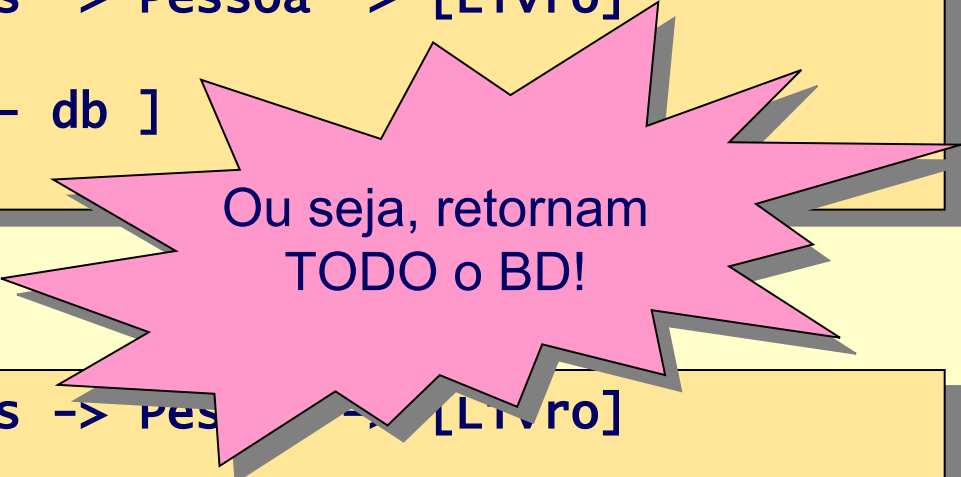
```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 livros db p =
3   [ liv | (p,liv) <- db ]
```

nova variável, com o mesmo nome

Erro de lógica comum

- A versão incorreta:

```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 livros db p =
3   [ liv | (p,liv) <- db ]
```



Ou seja, retornam
TODO o BD!

é equivalente a:

```
1 livros :: BancoDados -> Pessoa -> [Livro]
2 livros db p =
3   [ liv | (novo,liv) <- db ]
```